Practical 3

Max Heap Sort

Introduction

Heap Sort is a comparison-based sorting algorithm that uses a Binary Heap data structure. In Max Heap Sort, the largest element is placed at the root of the heap. The root is repeatedly swapped with the last element, and the heap is rebuilt until the array is sorted in ascending order.

Aim

To sort a list of elements in ascending order using the Max Heap Sort algorithm.

Objective

- To understand the concept of a Max Heap.
- To implement Heap Sort using a Binary Heap.
- To efficiently sort data with O(n log n) time complexity. Algorithm

1. Read the number of elements.
2. Store the elements in an array.
3. Build a Max Heap from the array.
4. Swap the root (maximum element) with the last element.
5. Reduce the heap size by one.
6. Heapify the root to restore the Max Heap.
7. Repeat Steps 4–6 until all elements are sorted.
8. Display the sorted array.

Pseudocode

START
Read n
Read array A

Build Max Heap
FOR i = n-1 DOWNTO 1

Swap A[0] and A[i]
Heapify(A, i, 0)
ENDFOR

Print A
STOP

Code:

```
def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2

    if left < n and arr[left] > arr[largest]:
        largest = left

    if right < n and arr[right] > arr[largest]:
        largest = right

    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

def heap_sort(arr):
    n = len(arr)


    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)


    for i in range(n - 1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]
        heapify(arr, i, 0)

n = int(input("Enter the number of elements: "))
arr = []

print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

heap_sort(arr)

print("Sorted array:")
print(arr)
```

```
Enter the number of elements: 5
Enter the elements:
88
55
44
33
6
Sorted array:
[6, 33, 44, 55, 88]
```

Advantages

- Efficient with O(n log n) time complexity.
- Performs sorting in-place (no large extra memory required).
- Suitable for large datasets.
- Worst-case performance is guaranteed.

Disadvantages

- Not a stable sorting algorithm.
- Slightly more complex than Bubble, Selection, and Insertion Sort.
- Can be slower than Quick Sort in practice due to cache performance.

Time Complexity

Case	Complexity
Best Case	O(n log n) Average Case O(n log n)
Worst Case	O(n log n)

Space Complexity

O(1) (In-place sorting)

Conclusion

Max Heap Sort is an efficient comparison-based sorting algorithm that uses a Max Heap to repeatedly place the largest element in its correct position. It guarantees O(n log n) time complexity in all cases and is well suited for sorting large datasets efficiently.